SOFTWARE DEVELOPMENT PROJECT (SDP)

Session 6 – UML Class Diagram Design

Class Diagram Design

ExamEnv: object-oriented analysis and UML class model

Project	ExamEnv – Infrastructure-as-Code for Exams
Document version	1.0
Date	29 October 2026
Group No.	Group 1
Team members	Gaurav Shukla – CloudOps and Agents Gaurav Singh – Backend and Research Manya Tiwari – Backend and Research Nandani – ML (Parser)
Course	Software Engineering / SDP – CA-2
Status	Draft for submission

1. Object-Oriented Analysis

Object-oriented analysis turns the requirements (Session 4) and use cases (Session 5) into a model of classes. The team used a noun-verb analysis: nouns in the SRS became candidate classes or attributes, and verbs became methods.

1.1 Class identification

Candidate (noun in SRS)	Decision
User, Teacher, Student, Admin	Classes. Teacher, Student and Admin specialise User (inheritance).
Exam	Class: groups documents, specs and statuses for one exam.
Requirement document (PDF)	Class RequirementDocument.
Parse job	Class ParseJob: tracks asynchronous Celery work.
ML Parser	Class MLParser: service that produces YAML.
Environment spec (YAML)	Class EnvironmentSpec.
Packages, ports, datasets / mounts	Classes PackageSpec, PortMapping, VolumeMount: parts of a spec (composition).
Student Agent	Class StudentAgent: CLI that provisions environments.
Container / environment	Class EnvironmentInstance.
Status (Ready / Failed / Not Started)	Class StatusRecord.
Verification log, error log	Class VerificationLog.
WebSocket updates	Class StatusBroadcaster.
Template Library	Class Template (future release).
Password, email, file name, time	Attributes, not classes.

1.2 Enumerations used as attribute types

Enumeration	Values
Role	ADMIN, TEACHER, STUDENT
ParseState / JobState	QUEUED, RUNNING, COMPLETED, FAILED
SpecState	DRAFT, PUBLISHED
ProvisionState	NOT_STARTED, IN_PROGRESS, READY, FAILED

2. UML Class Diagram

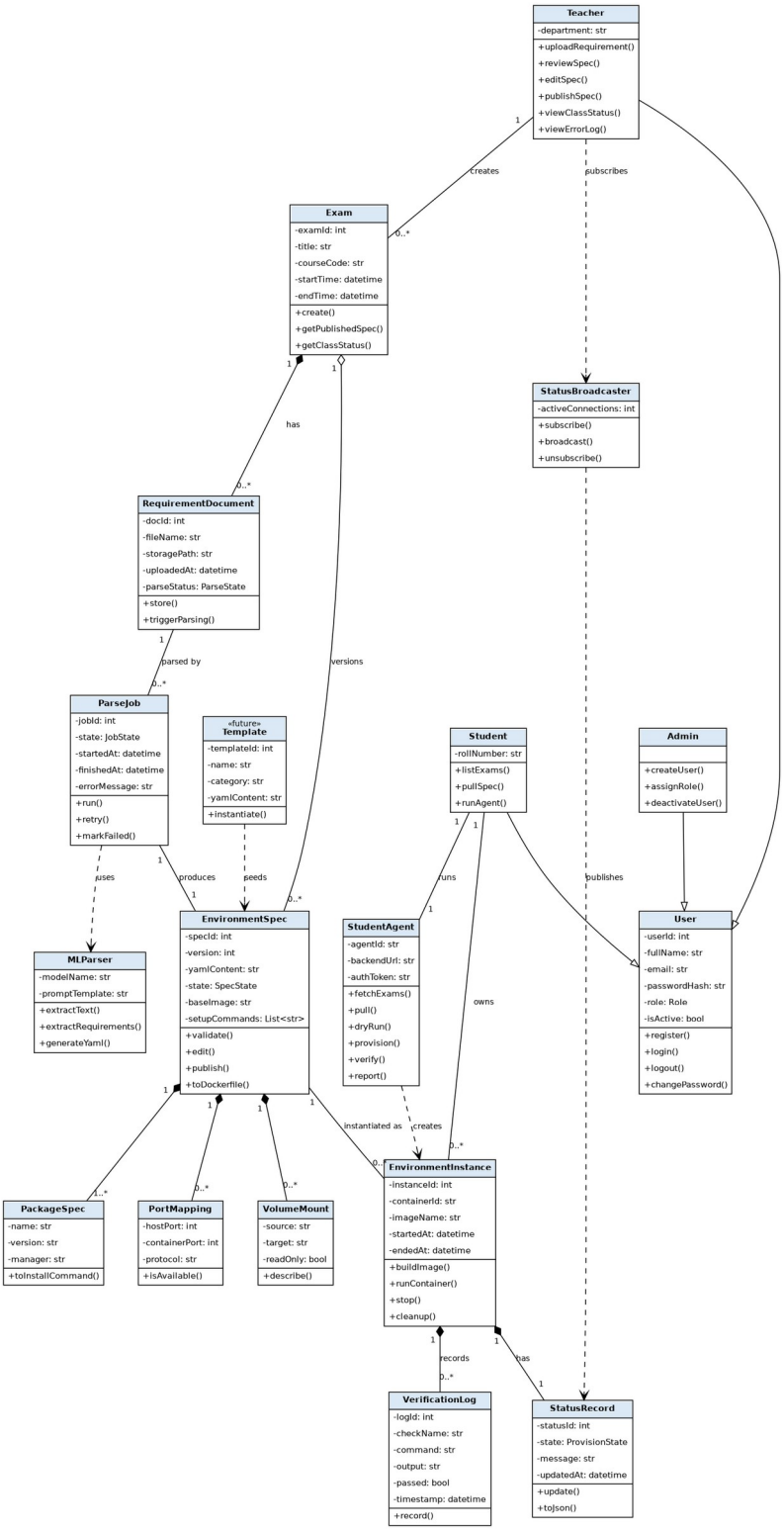


Figure 1: ExamEnv class diagram (17 classes). Full-size PNG: class_diagram.png. Editable: class_diagram.drawio.

3. Class Dictionary

Class	Responsibility	Main methods
User	Base class for all accounts: identity, credentials and role.	register, login, logout, changePassword
Teacher	Creates exams, uploads PDFs, reviews and publishes specs, monitors class.	uploadRequirement, reviewSpec, publishSpec, viewClassStatus
Student	Pulls specs and runs the agent.	listExams, pullSpec, runAgent
Admin	Manages accounts and roles.	createUser, assignRole, deactivateUser
Exam	One lab or exam with its time window; owns documents and specs.	create, getPublishedSpec, getClassStatus
RequirementDocument	Uploaded PDF and its storage details.	store, triggerParsing
ParseJob	Tracks one asynchronous parsing run, with retry.	run, retry, markFailed
MLParser	Extracts requirements from text and generates YAML.	extractText, extractRequirements, generateYaml
EnvironmentSpec	The Infrastructure-as-Code spec with version and Draft/Published state.	validate, edit, publish, toDockerfile
PackageSpec	One required package with version and package manager.	toInstallCommand
PortMapping	One exposed port mapping.	isAvailable
VolumeMount	One dataset or code mount.	describe
StudentAgent	CLI on the student machine: pull, dry-run, provision, verify, report.	pull, dryRun, provision, verify, report
EnvironmentInstance	One student's container built from a spec.	buildImage, runContainer, stop, cleanup
StatusRecord	Current readiness state of one instance.	update, toJson
VerificationLog	Result of one verification check.	record
StatusBroadcaster	Pushes status changes to dashboards over WebSockets.	subscribe, broadcast, unsubscribe
Template (future)	Pre-built verified YAML used to seed a spec.	instantiate

4. Relationships

Between	Type	Multiplicity	Meaning
Teacher, Student, Admin → User	Inheritance	-	Each is a kind of User.
Teacher – Exam	Association	1 to 0..*	A teacher creates many exams.
Exam ◆ RequirementDocument	Composition	1 to 0..*	Documents belong to one exam and are removed with it.
RequirementDocument – ParseJob	Association	1 to 0..*	A document can be parsed more than once (retry).
ParseJob → MLParser	Dependency	-	The job uses the parser.
ParseJob – EnvironmentSpec	Association	1 to 1	A successful job produces one draft spec.
Exam ◇ EnvironmentSpec	Aggregation	1 to 0..*	An exam has many spec versions that can exist on their own.
EnvironmentSpec ◆ PackageSpec / PortMapping / VolumeMount	Composition	1 to 1..* / 0..* / 0..*	The parts exist only inside a spec.
Student – EnvironmentInstance	Association	1 to 0..*	A student owns the instances built for them.
EnvironmentSpec – EnvironmentInstance	Association	1 to 0..*	One spec is instantiated for many students.
EnvironmentInstance ◆ StatusRecord / VerificationLog	Composition	1 to 1 / 0..*	Status and logs belong to one instance.
Student – StudentAgent	Association	1 to 1	A student runs one agent.
StudentAgent → EnvironmentInstance	Dependency	-	The agent creates instances.
StatusBroadcaster → StatusRecord	Dependency	-	Publishes status changes.
Teacher → StatusBroadcaster	Dependency	-	Teacher dashboard subscribes.
Template → EnvironmentSpec	Dependency	-	Seeds a draft spec (future).

5. Design Decisions

- Inheritance for users keeps shared login code in one class and role-specific actions in subclasses.
- EnvironmentSpec is composed of small classes so that the YAML can be validated field by field.

- ParseJob is separate from RequirementDocument so that failures and retries are tracked without changing the document.
- StatusRecord and VerificationLog are separate so the dashboard can show a quick status and open the detailed log only when needed (UC06 and UC07).
- The model matches the planned PostgreSQL tables: Users, Specs, Status and Logs.